

TDMA Schedule Optimizer: Design and Implementation Notes

Author: Athish M | Repository: tdma-schedule-optimizer | Package: tdma v1.0.0

1. The Problem in My Own Words

I built this optimizer to solve broadcast scheduling for radio nodes on a shared 2.4 GHz wireless channel. When nodes share a radio channel, uncontrolled transmissions cause packet collisions. Time Division Multiple Access (TDMA) fixes this by breaking time into repeating frames of fixed timeslots. Each node gets one or more slots to transmit.

The network sits on a 2D plane with an omnidirectional radio range of 500.0 meters. A valid schedule must handle two collision types:

- **Distance-1 collision:** Two nodes within 500 meters cannot transmit in the same slot. If they transmit together, each radio drowns out the other and neither can receive.
- **Distance-2 collision (the hidden-terminal problem):** Two nodes might be more than 500 meters apart, but share a common neighbor within 500 meters of both. If both transmit at the same time, the neighbor hears overlapping signals and receives garbage.
- **Spatial reuse:** Two nodes that are 3 or more hops apart in the network graph can transmit in the same timeslot. No shared receiver can hear both, so their transmissions do not collide.

My goal is to find a conflict-free schedule that uses the minimum number of timeslots. A shorter frame means every node transmits more frequently, latency drops, and overall network throughput increases.

2. How I Modelled It

I modelled the radio network as an undirected physical graph $G = (V, E)$. The vertices V are radio transceivers with coordinates (x, y) . An edge exists between node u and node v if their Euclidean distance satisfies $d(u, v) \leq 500.0$ m. I treat a distance of exactly 500.0 m as in range, applying a numerical tolerance of 10^{-9} meters to avoid floating-point roundoff issues.

To handle both 1-hop and 2-hop conflicts directly, I construct the squared graph G^2 . The graph G^2 has the same vertices as G . An edge exists between u and v in G^2 if the shortest path distance in G is 1 or 2 hops: $1 \leq \text{dist}_G(u, v) \leq 2$.

With this construction, a valid TDMA schedule is equivalent to a proper vertex colouring of G^2 . Two nodes that share an edge in G^2 conflict and must receive different colors (slots). Two nodes with no edge in G^2 are at least 3 hops apart and can safely share a slot.

Vertex colouring on arbitrary graphs is NP-hard. Even for unit-disk graphs, distance-2 colouring remains NP-hard. I use two graph properties to bound the required slots:

- **Lower bound:** The maximum clique size $\omega(G^2)$. If a group of nodes all pairwise conflict within 2 hops, every node in that group needs a distinct slot.
- **Upper bound:** The maximum vertex degree $\Delta(G^2) + 1$, achievable by greedy colouring.

3. What I Tried and What Happened

I implemented five heuristic algorithms to find schedules quickly, and two exact solvers to verify whether those schedules reached the mathematical minimum.

3.1 Five Heuristics

I evaluated all five heuristics on the 4x4 grid topology (16 nodes, 300 m spacing, 500 m range):

1. **Largest-Degree-First (LDF / Welsh-Powell):** Sorts nodes descending by degree in G^2 and colors greedily. It assigned 9 slots in 0.06 ms. It was the fastest heuristic.
2. **DSATUR (Brélaz):** Selects the uncolored vertex with the highest number of distinct colors among its neighbors. It assigned 9 slots in 0.25 ms.
3. **Smallest-Last (Matula and Beck):** Repeatedly removes the minimum-degree vertex from the remaining subgraph, then colors in reverse order. It assigned 9 slots in 0.09 ms.
4. **Randomized Restarts:** Evaluates 1000 seeded random vertex permutations with greedy first-fit. It consistently found 9 slots in 60.8 ms.
5. **Local Search (Color Reduction):** Starts from the best greedy schedule and attempts to eliminate the highest slot using Kempe-chain swaps and tabu search. On the 4x4 grid, it attempted to reduce 9 slots to 8, but correctly stopped because 8 slots is mathematically impossible.

All five heuristics matched the theoretical minimum of 9 slots on the 4x4 grid. LDF proved to be the fastest option.

3.2 Two Exact Solvers

Heuristics cannot prove optimality on their own. I added two exact solvers to certify the true minimum frame length:

1. **Google OR-Tools CP-SAT:** Formulates the problem as constraint optimization with binary assignment variables $x_{v,c}$ and slot indicators y_c . I broke color permutation symmetry by pre-colouring a maximum clique in G^2 . It solved the 4x4 grid in 0.60 ms, confirming that 9 slots is the exact optimum.
2. **Pure-Python Branch-and-Bound Fallback:** A zero-dependency backtracking solver. It uses clique pre-colouring, lower-bound pruning, and DSATUR variable ordering. It serves as a standalone fallback when OR-Tools is not installed.

3.3 What I Considered but Rejected

During design, I considered several alternative optimization approaches:

- **Genetic Algorithms:** I rejected genetic algorithms because their stochastic search provides no guarantee of optimality. Furthermore, distance-2 graph coloring has strict hard constraints, and random crossover or mutation operators frequently produce invalid schedules that require expensive repair routines.
- **Simulated Annealing:** I rejected simulated annealing because penalty tuning on soft conflict formulations is fragile. It often converges to near-valid states with residual collisions, requiring an auxiliary deterministic coloring pass.
- **Plain ILP without Symmetry Breaking:** A naive ILP formulation assigns colors $0 \dots K-1$. Because any permutation of slot assignments is equivalent, the solver explores thousands of symmetric branches. Anchoring a maximum clique upfront breaks this symmetry and allows CP-SAT to solve the 4x4 grid in under 1 millisecond.

3.4 Summary of Design Decisions

Why graph squaring? Squaring isolates distance constraints into edge adjacency. Standard graph coloring routines can then run unmodified.

Why five heuristics? They offer distinct trade-offs between execution speed and search depth across irregular topologies.

Why an exact solver? Exact methods certify whether heuristic schedules reached the mathematical floor.

Why a separate verifier? A separate verifier independently evaluates physical invariants using BFS on G .

4. Results on the Benchmark Topologies

I tested the optimizer across four representative topologies. Every topology matched the exact theoretical optimum with zero heuristic gap.

Topology Scenario	Nodes	G Edges	G ² Edges	Max Deg	Exact Opt	Best Heur	Gap	Runtime
1. 4x4 Grid (300 m)	16	42	90	15	9 slots	9 slots	0	0.48 ms
2. Sparse Linear (350 m)	16	15	29	4	3 slots	3 slots	0	0.17 ms
3. Dense Cluster (≤500 m)	16	120	120	15	16 slots	16 slots	0	0.22 ms
4. Disconnected Clusters	16	56	56	7	8 slots	8 slots	0	0.30 ms

4.1 Detailed Look at the 4x4 Grid

Why does the 4x4 grid with 300 m spacing require exactly 9 slots?

- Horizontal and vertical neighbors sit 300 m apart, within the 500 m radio range.
- Diagonal neighbors sit $\sqrt{300^2 + 300^2} \approx 424.26$ m apart, also within the 500 m range.
- Any 3x3 block of 9 nodes has a maximum path distance of 2 hops in G .
- Therefore, all 9 nodes in any 3x3 block conflict with each other in G^2 , forming a clique of size 9: $\omega(G^2) \geq 9$.
- Because the chromatic number $\chi(G^2) \geq \omega(G^2)$, no schedule can use fewer than 9 slots.

Spatial reuse occurs at the edges and corners. The four corner nodes (Node_01 at (0,0), Node_04 at (900,0), Node_13 at (0,900), and Node_16 at (900,900)) sit 3 or more hops apart. The optimizer groups all four corner nodes into Slot 8.

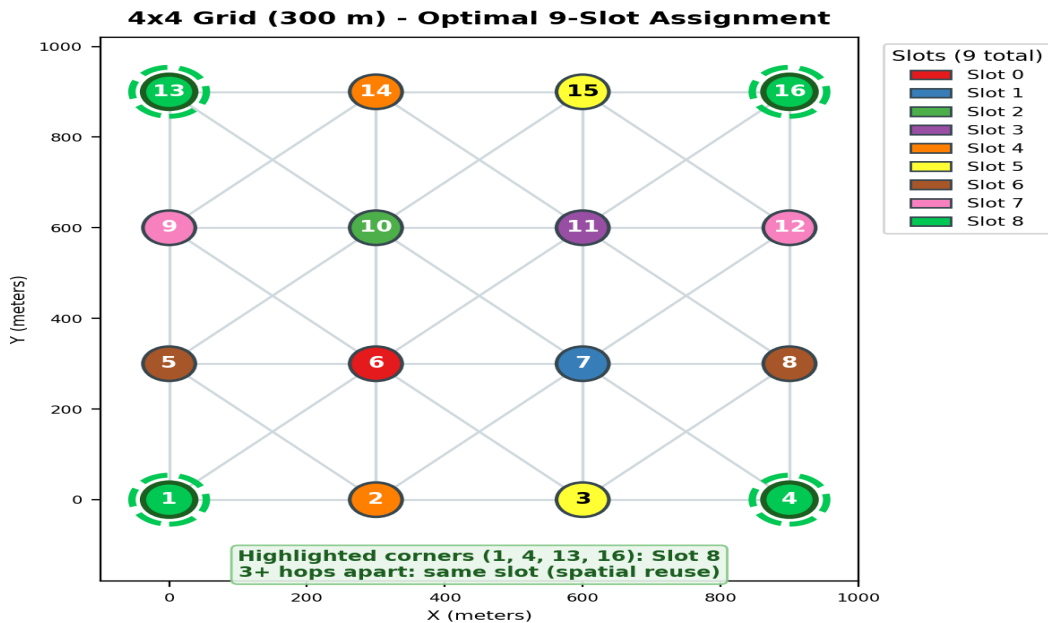


Figure 1: Verified 9-slot schedule for the 4x4 grid (300 m spacing). Corner nodes 1, 4, 13, 16 share Slot 8 (spatial reuse).

4.2 Real CLI Output for the 4x4 Grid

Below is the complete, raw terminal output generated by executing `python -m tdma.cli --coords-file examples/grid_4x4_300m.json`. The report displays total nodes, configured range, optimized frame length, individual node allocations, and the structural slot-by-node boolean schedule matrix:

```
=====
TDMA TOPOLOGY OPTIMIZATION REPORT
=====
Total Nodes Processed : 16
Configured Radio Range : 500.0 meters
Optimized Frame Length : 9 unique timeslots (Lower is better)
-----
NODE -> SLOT ASSIGNMENTS:
Node_01: Slot 8
Node_02: Slot 4
Node_03: Slot 5
Node_04: Slot 8
Node_05: Slot 6
Node_06: Slot 0
Node_07: Slot 1
Node_08: Slot 6
Node_09: Slot 7
Node_10: Slot 2
Node_11: Slot 3
Node_12: Slot 7
Node_13: Slot 8
Node_14: Slot 4
Node_15: Slot 5
Node_16: Slot 8

STRUCTURAL TDMA SCHEDULE MATRIX (Slot x Node Boolean Matrix):
Slot \ Node | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16
-----
Slot 00      | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
Slot 01      | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
Slot 02      | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0
Slot 03      | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0
Slot 04      | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0
Slot 05      | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0
Slot 06      | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0
Slot 07      | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0
Slot 08      | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1
-----
Execution finalized cleanly. Schedule verified conflict-free.
=====
```

5. How I Checked the Answer

I wrote the verifier in `src/tdma/verify.py` as an isolated module. It does not import any graph coloring code or the G^2 conflict graph.

5.1 Verification Invariants

The verifier takes the physical graph G and the schedule dictionary. It checks three invariants:

1. **Complete Coverage:** Every node in V appears in the schedule.
2. **Contiguous Indexing:** Slot numbers form a contiguous range $0 \dots K-1$ with no empty gaps.
3. **Conflict Freedom:** For every pair of nodes (u, v) sharing a slot, the verifier computes the shortest path length in G using Breadth-First Search (BFS). It asserts that $\text{dist}_G(u, v) \geq 3$.

If any two nodes sharing a slot are 1 hop apart, the verifier reports a direct collision. If they are 2 hops apart, it identifies their shared intermediate neighbor and reports a hidden-terminal violation.

5.2 Boundary Rule and Numerical Tolerance

A distance of exactly 500.0 m counts as in range (tolerance $1e-9$ m). This boundary rule ensures consistent edge creation between topology generation, optimization, and verification, preventing floating-point precision differences from dropping edge adjacencies.

5.3 Distinguishing Correctness from Optimality

The verifier checks conflict freedom and structural correctness. It does not prove optimality on its own. Optimality is established by the 9-clique lower bound and certified by the exact solvers. A schedule with 16 slots on the 4x4 grid would pass the verifier with zero conflicts, but it would not be optimal. Optimality requires comparing the verified frame length against the clique lower bound $\omega(G^2)$ and the CP-SAT objective.

5.4 Automated Test Suite

The test suite contains 47 automated tests covering edge cases, property bounds, and round-trip translations. All 47 tests pass under `pytest`. Key test categories include:

- Single-node topologies (1 slot, 0 edges).
- Co-located coordinate detection and immediate error termination.
- Disconnected components and multi-cluster spatial reuse.
- Intentional conflict injection to confirm the verifier catches 1-hop and 2-hop violations.
- XML round-trip parity checks for the EMANE bridge.

6. Part 2: EMANE Integration and Emulation Bridge

Part 2 bridges the Python optimizer to the Extendable Mobile Ad-hoc Network Emulator (EMANE). EMANE is a real-time framework that emulates link and physical layer radio behaviors in network research.

6.1 Architecture Overview

Part 2: EMANE Integration Architecture

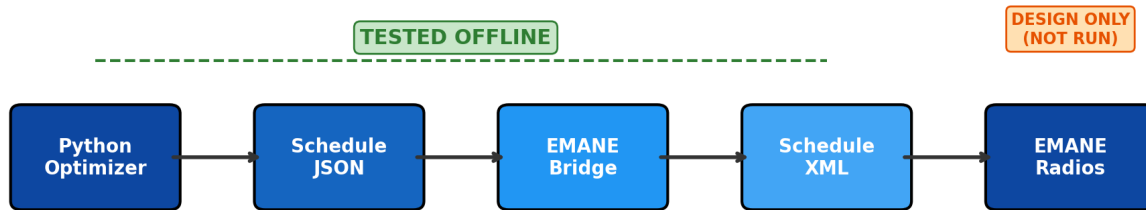


Figure 2: EMANE integration pipeline. Schedule translation is tested offline; live RF emulation inside Linux network namespaces is design-only.

6.2 What I Tested Offline vs What Was Not Run

- **Tested offline:** I tested JSON schedule parsing, 1-based NEM identifier mapping, EMANE XML schedule generation, round-trip XML schema validation, and Python event injection script generation. All offline bridge tests pass under pytest without requiring EMANE.
- **Not run:** I did not run live over-the-air packet emulation inside Linux network namespaces with EMANE daemons. Running live EMANE requires root privileges, Linux kernel virtual interfaces, and a containerized test environment.

6.3 Radio Propagation and Range Modeling in EMANE

EMANE's TDMA scheduler radio model has no built-in 500-meter cutoff. Physical reachability is determined by node positions, RF pathloss events (PathlossEvent, Event ID 101), antenna gain, and receiver sensitivity. If EMANE runs without location or pathloss events, all 16 virtual radios share a single broadcast domain over multicast OTA (224.1.2.8:45703). Under global visibility, transmissions from nodes sharing Slot 8 (such as Node_01 and Node_04) would collide at the physical layer. To demonstrate spatial reuse in emulation, pathloss and transmit power must be calibrated so received signal strength drops below detection threshold beyond 500 meters.

6.4 Official EMANE Verification Log

Reference repositories: <https://github.com/adjacentlink/emane> | <https://github.com/adjacentlink/emane-guide>

Parameter / Component	Category	Verified Name / Setting	Source file & quoted line	Status
MAC Model Library	MAC Plugin	tdmaeventschedulerradiomodel	tdmaradiomodel.xml.in: <code><mac library='tdmaeventschedulerradiomodel'></code>	VERIFIED
MAC vs Structure	Architecture	<structure> holds timing	tdma-radio-model.txt: <i>'The TDMA structure defines: Slot size..., Slot overhead...'</i>	VERIFIED
MAC PCR Curve URI	MAC Param	pcrcurveuri	tdmaradiomodel.xml.in: <code><param name='pcrcurveuri' value='...tdmabasemodelpcr.xml'/></code>	VERIFIED
MAC Queue Controls	MAC Param	queue.depth, aggregation, etc.	tdmaradiomodel.xml.in: <code><param name='queue.depth' value='255'/></code>	VERIFIED
Schedule Root Element	XML Schema	<emane-tdma-schedule>	tdmaschedule.xsd: <code><xs:element name='emane-tdma-schedule'></code>	VERIFIED
Structure Element	XML Element	<structure frames=.. slots=..>	tdmaschedule.xsd: <code><xs:element name='structure' slotduration=..></code>	VERIFIED
Multiframe / Frame	XML Element	<multiframe> / <frame>	tdmaschedule.xsd: <code><xs:element name='multiframe'> containing <frame></code>	VERIFIED
Slot Allocation	XML Element	<slot index=.. nodes=..><tx/>	tdmaschedule.xsd: <code><xs:element name='slot'> with <tx>, <rx>, <idle></code>	VERIFIED
CLI Injection Tool	CLI Script	emaneevent-tdmaschedule	tdma-radio-model.txt: <code>\$ emaneevent-tdmaschedule schedule.xml -i lo</code>	VERIFIED
Python Event Class	Python API	emane.events.TDMAScheduleEvent	tdmascheduleevent.py: <code>class TDMAScheduleEvent; def structure(..); def append(..)</code>	VERIFIED
Python Event Service	Python API	emane.events.EventService	eventservice.py: <code>class EventService(eventchannel); def publish(nemId, event)</code>	VERIFIED
MAC Param Schedule	MAC Param	<param name='schedule'>	Not in official tdmaradiomodel.xml.in ; runtime events appear required	ASSUMPTION

6.5 Planned Test Plan (Not Yet Run)

1. **Valid schedule test:** Transmit ping and iperf packets across all pairs. Verify that packet delivery occurs strictly inside allocated 1.0 ms slots at 9 ms frame intervals, with expected delivery only in scheduled slots.
2. **Conflicting schedule test:** Intentionally assign two 2-hop neighbors to the same slot. Verify that simultaneous transmissions cause collision, low SINR, and packet drops at the intermediate receiver.
3. **Range calibration test:** Tune the pathloss model so communication drops off at approximately 500 meters, verifying spatial reuse without false collisions.

7. Things I Am Unsure About

I identified two areas where information is incomplete:

1. **Schedule loading via MAC parameter:** The official `tdmaradiomodel.xml.in` manifest does not list a schedule parameter. Official guides inject schedules dynamically using `emaneevent-tdmaschedule` or the `TDMAScheduleEvent` API. Loading a schedule statically from a MAC parameter remains an unverified assumption.
2. **The brief's 5-slot sample:** The assignment brief showed a sample output with 5 slots, but did not provide coordinates. On a 4x4 grid with 300 m spacing and 500 m radio range, 5 slots is mathematically impossible due to the 9-clique lower bound. If 5 slots was intended, either the radio range was lower (excluding diagonals) or the topology was linear or sparse. I treated the brief's sample as an illustrative format example rather than a fixed target. The independent verifier checks physical correctness.

8. Next Steps

If I continue work on this project, I will:

1. **Execute live EMANE on a Linux testbed:** Set up Linux network namespaces and run EMANE daemons in a Docker container to measure real packet latency and throughput.
2. **Calibrate propagation models:** Tune freespace or two-ray pathloss parameters to produce the 500-meter cutoff in live emulation.
3. **Multi-channel TDMA:** Extend the optimizer to allocate both time slots and frequency channels (t, f) , reducing frame length in dense networks.
4. **Dynamic mobile scheduling:** Adapt the scheduler for moving nodes using incremental coloring and distributed slot reservations.